



REDLINE DAYLIGHT • AUTOMATION HANDBOOK

The Complete Guide to Learning Ansible

Your complete reference to **Ansible** — the Architecture, installing on every OS, setting up SSH Keys, Modules and the Inventory, Ad-hoc Commands, YAML fundamentals, full Playbooks with result reading, and a deep dive into Roles, Templates, Handlers and the Galaxy platform. Based on the course by the **Cloud Code** channel — transcribed, corrected, expanded, and organized into a complete written reference.

By **Nabawy**

AUTHOR & COMPILER

REDLINE EDITION

Daylight • A4-Fixed

SSH

Secure Access

YAML

Playbook Language

Roles

Deep Dive 🐙

Table of Contents

	The Map — Executive Summary	3		Ad-hoc Commands — Instant Execution	12
	Intro — What is Ansible?	4		YAML Basics — The Language	13
	Architecture — Two Components	5		Playbooks — Write & Run	14
	Comparison — Manual vs Bash vs Ansible	6		Playbooks — Removal & Results	15
	Use Cases — Where Ansible Shines	7		Roles — The Concept & Anatomy	16
	Installation — Every OS	8		Templates & Handlers	17
	SSH Keys — Passwordless Access	9		Ansible Galaxy — Use & Share	18
	Modules — The Execution Units	10		Test Yourself — 12 Questions	19
	Inventory — Your Server Map	11		Answer Key	20
				Cheat Sheet	21

► **TASK [Status Color Legend]** how to read results

-  **ok** — server already in the desired state, no change  **changed** — a change was applied successfully
-  **failed** — a task errored out  **RED** — headings & structure (visual identity)

[**DEBUG**] **How to use this book:** 12 connected chapters — each opens from the previous and sets up the next. The Roles topic gets a full three-page deep dive (concept → templates & handlers → Galaxy hands-on), because that's where beginners stall. Every line from the source course is here — compare freely.



The Map — The Whole Route

Before touching a single command, here's the whole story — every sentence below is a link that takes you to its chapter.

We start with an existential question: **why do we bother logging into every server manually?** The answer begins in [Chapter 1 — Intro](#): Ansible is an **Automation** tool, **Open-Source**, **Agentless**, with simple **YAML** syntax, a **Declarative** approach, and safe repeatability — **Idempotent**. Then in [Chapter 2 — Architecture](#) we see the whole picture in just two components: a **Control Node** whispering over an **SSH Connection** with a **Public/Private Key Pair** to the **Managed Nodes** (Ubuntu / CentOS / DB).

In [Chapter 3 — Comparison](#) we weigh Ansible against the Manual way and Bash scripts on 3 different-distribution servers to install Nginx. In [Chapter 4 — Use Cases](#) we tour the three domains: **Configuration Management**, **Infrastructure Provisioning**, and **Network Automation**.

Now the practical gear: [Chapter 5 — Installation](#) on every OS, then [Chapter 6 — SSH Keys](#) — the foundation every automation stands on. In [Chapter 7 — Modules](#) we meet the execution units, in [Chapter 8 — Inventory](#) we map our servers, and in [Chapter 9 — Ad-hoc Commands](#) we execute instantly.

Language and play: [Chapter 10 — YAML](#), then [Chapter 11 — Playbooks](#) with a complete Apache build, and [Chapter 11-b](#) with removal and result reading. Then the deep dive: [Chapter 12 — Roles](#) (the concept and anatomy), [Chapter 12-b — Templates & Handlers](#) (the two folders everyone asks about), and [Chapter 12-c — Galaxy](#) (using and sharing roles). Close with the [Quiz](#) and the [Cheat Sheet](#). The route is clear. ►

KEY IDEA **The unifying idea:** you write the Desired State once — Ansible enforces it across all your Managed Nodes. And when your automation grows, Roles give it structure, Templates give it flexibility, and Galaxy gives it a community.

Intro – What is Ansible?

Ansible is an **Open-Source Automation** tool used primarily for two things:

1 Configuration Management

Managing settings — installing and updating software and configuring files across your servers.

2 Infrastructure Provisioning

Building and preparing infrastructure — creating servers and cloud resources.

► **TASK [Ansible's Four Core Features]** the four pillars

1.1 Agentless

- 📦 No need to install any software or **Agents** on the target **Managed Nodes**.
- 🔑 Connection happens over **SSH** (on Linux systems) or **WinRM/SSH** (on Windows systems).
- 🐍 The only requirement on Linux servers is a **Python** environment.

1.2 Simple Syntax

All configurations and automation scenarios are written in **YAML** — a language that is very simple to read and write, easy to learn and execute. No complex brackets, no semicolons; just spaces and order.

1.3 Declarative Approach

You define the **Desired State** in the YAML file — like: "I want Apache and Git installed" — and Ansible executes the steps needed to reach that state on its own. It also supports the **Imperative Approach** when using **Ad-hoc Commands**.

1.4 Idempotency

Running the same script multiple times in a row **changes nothing unless a change is actually needed**.

[EXAMPLE] If you ask Ansible to install Apache and it's already installed at the required version → it won't reinstall it and won't touch the server. The status shows **ok**. Not installed? It installs it → **changed**.

Agentless**Simple YAML****Declarative****Idempotent**

▶▶ How is this tool built internally? Two components only — **Chapter 2: Architecture**



Architecture – Two Components

Ansible's architecture consists of **two main components** — deliberate simplicity that is the secret of its power.

[Architecture – the two components and the channel]



The channel: a **Public/Private Key Pair** — secured connection, no passwords.

2.1 Control Node

The machine or server where **Ansible** is installed, and through which the **DevOps** engineer runs commands and **Playbooks**. It can be a personal laptop or a Linux server in a lab or the cloud.

2.2 Managed Nodes (Targets)

The servers targeted for applying changes and configurations. They receive instructions over **SSH** and execute them **without needing Agents installed** — that's why the architecture is called Agentless.

[**KEY IDEA**] Notice what doesn't exist: no central database, no daemon constantly running on the targets, no agent to update. Ansible connects when needed, executes, then leaves. The footprint on Managed Nodes = zero.

► TASK [Component Map] who does what

Component	What is it?	What's installed on it?	Its role
Control Node	Laptop / Linux Server	Ansible (only)	Running commands and Playbooks
Managed Nodes	Ubuntu / CentOS / DB...	Nothing — Python only	Receiving SSH commands & executing

[**WATCH OUT**] Common beginner mistake: installing Ansible on the Managed Nodes "just in case." Don't — they don't need it at all. The Control Node is the only place installation belongs.

🎓 **EXAM** Interview question: "Describe Ansible's architecture?" → **Two components**: a Control Node (has Ansible, runs everything) + Managed Nodes (receive over SSH, no Agents), and the channel between them is a Key Pair.

🔗 Architecture clear — but how does it beat traditional methods? → [Chapter 3: Comparison](#)



Comparison – Manual vs Bash vs Ansible

Imagine you're an engineer managing **3 Linux servers** with different distributions — **Ubuntu, CentOS, Alpine** — and you want to install **Nginx** on all of them. Three paths, and the difference is a whole philosophy:

Aspect	👉 Manual	📄 Bash Scripts	▶ Ansible
How it works	SSH into every server and run commands by hand.	Write a Bash script that detects the OS and runs the commands.	Write one simple YAML file defining the Desired State for everyone.
Speed & effort	Slow and full of mistakes.	Centralized but requires high scripting expertise.	Fast, centralized, and Simple Syntax.
Idempotency	Not available — manual double-checking.	Not automatic — conditions written by hand.	Automatically supported ✓
Distribution support	Requires knowing every distro's commands (<code>apt</code> , <code>yum</code> , <code>apk</code>).	Requires writing <code>if/else</code> for every OS.	Modules handle all distros automatically ✓

[ASIDE • The paradox] Bash scripts are smarter than Manual — but they accumulate: every new distribution = a new `if/else` branch. Ansible flips the equation: you don't write **how** to execute — you write **what** you want, and it knows the How for every distro inside the Modules.

► TASK [Reading the table practically] real-world lens

By real standards: Manual needs 3 SSH sessions and 3 different commands (`apt install` / `yum install` / `apk add`). Bash: one script with branching per package manager. Ansible: **one Task** — `package: name=nginx state=present` — works on all three, with Idempotency verifying itself on every future run.

[🎓 EXAM] Memorize the pairs: Manual = slow + errors + no Idempotency · Bash = centralized but heavy scripting · Ansible = **centralized YAML + automatic Idempotency + distro-aware Modules**. This exact comparison is a recurring interview question.

▶▶ Convinced — where is it actually used? → [Chapter 4: Use Cases](#)

Use Cases – Where Ansible Shines

Three major arenas where Ansible works today — the first is the most famous, the second carries a well-known industry debate.

4.1 Configuration Management

The most famous and important use: installing and updating **Packages**, adjusting **Files**, and updating Servers from one place — whether the servers sit in a local Datacenter or on the Cloud. Hundreds of servers, one command.

4.2 Infrastructure Provisioning

Creating and destroying cloud resources (**EC2, S3, Containers**) through the **APIs** of cloud services (AWS, Azure, GCP).

[ASIDE • Industry note] Many prefer **Terraform** for Provisioning and **Ansible** for Configuration Management. Division of labor: the contractor builds the house, the technician wires the electricity. 🏠🔧

4.3 Network Automation

Configuring and managing **Routers, Switches**, and **Firewalls** programmatically through automation protocols — instead of logging into every network device by hand.

► **TASK [The Quick Decision Map]** which tool when?

Your task	Right tool	Why
Install Nginx on 20 servers	Ansible	Configuration inside machines — its first domain
Create a VPC and 10 instances on AWS	Terraform	Cloud provisioning over API
Full project: build + configure	Both 🍷	Terraform builds → Ansible configures
Update firmware on network switches	Ansible	Network Automation — its third domain

🎓 **EXAM** Memorize the trio in order: **Configuration Management** (most famous) · **Infrastructure Provisioning** (cloud) · **Network Automation** (routers/switches/firewalls). "What are Ansible's use cases?" — this is the model answer.

🔗 Domains are clear — now the gear: → **Chapter 5: Installation**



Installation – Setup on Every OS

Golden rule: Ansible is installed **only on the Control Node** — a rule with no exceptions. Pick your path by OS:

macOS — Homebrew

```
$ brew install ansible
```

Ubuntu / Debian — apt

```
$ sudo apt update && sudo apt install ansible -y
```

Any OS — Python Pip

```
$ pip3 install ansible
```

Windows — WSL

Use **WSL** to run Ubuntu inside Windows and install Ansible on it naturally — then apply the Ubuntu command above.

▶ TASK [Verify the Installation] it works

```
$ ansible --version
ansible [core 2.16.3]
  config file = /etc/ansible/ansible.cfg
  python version = 3.10.12
```

▶ TASK [Recommended VS Code Extensions] editor setup

[YAML Extension – Red Hat](#)[Ansible Extension – Red Hat](#)

[**WATCH OUT**] Common mistake: installing Ansible on the Managed Nodes "just in case." Useless — they only receive commands over SSH. Installation belongs to the Control Node alone.

[**DEBUG**] `pip3 install` is the shared option for every OS — especially useful if your distro's package manager ships an outdated version.

 **EXAM** Three install methods + Windows: `brew` · `apt` · `pip3` · **WSL**. And "Where is Ansible installed?" — **the Control Node only**.

 Tool ready — connections need securing: → [Chapter 6: SSH Keys](#)



SSH Keys – Passwordless Secure Setup

For the Control Node to connect to the Managed Nodes and execute instructions **automatically**, the connection should be set up over SSH keys — the **Public/Private Key Pair**.

[Play – three steps and the connection is done]

1 **ssh-keygen**
generate the Pair



2 **ssh-copy-id**
push the Public Key



3 **Test**
no Password ✓

6.1 Generate the Key Pair on the Control Node

```
$ ssh-keygen -t rsa -b 4096
```

This creates two files in the `~/.ssh/` folder:

🔑 **id_rsa — Private Key**

The **Private Key** — **kept strictly secret** on the Control Node, never shared.

🔑 **id_rsa.pub — Public Key**

The **Public Key** — this is the one that **gets pushed** to the Managed Nodes.

6.2 Copy the Public Key to the Managed Node

```
$ ssh-copy-id -i ~/.ssh/id_rsa.pub ubuntu@<MANAGED_NODE_IP>
```

6.3 Test the Passwordless Connection

```
$ ssh ubuntu@<MANAGED_NODE_IP>
Welcome to Ubuntu 22.04 LTS ... # no password asked ✓
$ exit
```

[**DEBUG • AWS EC2**] On AWS EC2 machines, don't generate a new key — use the `.pem` Private Key file provided by AWS for the first connection (User is usually `ubuntu` or `ec2-user`).

[**WATCH OUT**] Two common mistakes: copying `id_rsa` (Private) instead of `id_rsa.pub` (Public) = connection fails. And pushing the Private over insecure channels = a complete security hole. The Public travels; the Private never moves.

🎓 **EXAM** The literal sequence: `ssh-keygen -t rsa -b 4096` ← `ssh-copy-id -i ~/.ssh/id_rsa.pub ubuntu@IP` ← test `ssh ubuntu@IP`. From this moment, every Ansible command is completely silent.

▶ Channels open — now Ansible's tools and map: **Chapter 7: Modules**

Modules – The Execution Units

All of Ansible's work depends on **Modules** — ready-made software tools that perform specific functions. You don't write scripts; you call Modules and hand them the desired State.

► **TASK [The most famous core Modules]** know these five

Module	What it does
<code>apt</code> / <code>yum</code>	Managing, installing, updating, and removing Packages (depending on the distribution).
<code>ping</code>	Testing connectivity and readiness between the Control Node and the Managed Nodes.
<code>service</code>	Managing Services — start, stop, restart.
<code>copy</code>	Copying files from the Control Node to the Managed Nodes.
<code>command</code> / <code>shell</code>	Running direct system commands on the servers.

Hundreds of Modules cover almost everything — always refer to the official **Ansible Documentation** to find the right Module and its options for each task.

► **TASK [The three Modules you'll use first]** start here

ping

The first Module you'll try — proves the connection works. The success response is always: `"pong"`.

apt / yum

The heart of package management. With `state=present` it becomes Idempotent automatically.

service

Controls a Service's life: `started`, `stopped`, `restarted` — plus `enabled` at boot.

[**KEY IDEA**] The Module hides the How: you say `apt: name=nginx state=present` — and it knows whether to actually run `apt-get install` or skip because everything already exists. This is the root of Idempotency.

[**WATCH OUT**] `command` and `shell` can be a trap: they let you bypass all of Ansible's elegance and write raw non-Idempotent commands. Use them as a last resort when no dedicated Module exists.

🔑 **EXAM** Memorize the core five: `apt/yum` (Packages) · `ping` (Connectivity) · `service` (Services) · `copy` (Files) · `command/shell` (Direct commands).

🔗 Tools ready — now the target list: → [Chapter 8: Inventory](#)

Inventory – Your Server Map

The **Inventory** file is the file that holds the **IPs** of the Managed Nodes — divided into Groups that you target in commands.

► TASK [Example hosts.ini file] inventory example

```
# hosts.ini
[web]
192.168.1.101 ansible_user=ubuntu
192.168.1.102 ansible_user=ubuntu

[db]
192.168.1.201 ansible_user=ubuntu
```

[**DEBUG · File anatomy**] `[web]` and `[db]` = group names in square brackets · the lines under them = member IPs · `ansible_user=ubuntu` = the SSH user used to connect to that server — defined once here instead of repeated in every command.

► TASK [How do we target?] targeting

`[web]` → 2 Servers `[db]` → 1 Server `all` → 3 Servers

`ansible web ...` hits the web servers only · `ansible all ...` hits everyone · target by group instead of stuffing IPs manually.

[**KEY IDEA**] The Inventory is the "source of truth" for your infrastructure map: added a server? One line. Changed a user? One edit. Every command and Playbook coming next refers to this file via `-i hosts.ini`.

[**WATCH OUT**] Group names are strictly case-sensitive: `[web]` is not `[webs]` and not `[Web]`. One wrong letter in `targeting` = "no matching hosts" — and the error message won't hint at your typo.

🎓 **EXAM** Inventory = a file holding Managed Node IPs divided into Groups · Syntax: `[group_name]` then the lines · `ansible_user` defines the SSH user · always passed with `-i`.

▶▶ We have Modules and Inventory — real execution, no file needed: → [Chapter 9: Ad-hoc Commands](#)

⚡ Ad-hoc Commands – Instant Execution

Ad-hoc Commands run a **quick task without writing a full Playbook** — a single line straight from the terminal. This is the Imperative side we mentioned in Chapter 1.

► TASK [The general syntax] anatomy

```
ansible <group_or_all> -i <inventory_file> -m <module_name> -a "<module_arguments>"
```

► TASK [Three practical examples] from ping to install

1 Test connectivity — ping all servers:

```
$ ansible all -i hosts.ini -m ping
192.168.1.101 | SUCCESS => { "ping": "pong" }
192.168.1.102 | SUCCESS => { "ping": "pong" }
192.168.1.201 | SUCCESS => { "ping": "pong" }
```

2 Get the Uptime of web servers:

```
$ ansible web -i hosts.ini -a "uptime"
```

[**DEBUG**] When `-m` is omitted, the `command` Module is used **by default** — that's why we wrote `uptime` directly in `-a`.

3 Ensure Nginx is installed on all servers:

```
$ ansible all -i hosts.ini -m apt -a "name=nginx state=present" --become
```

[**KEY IDEA**] `--become` = root privileges (like `sudo`) — installation always needs them. And `state=present` is the language of Idempotency: "make sure it exists" — already exists? Nothing happens.

[**WATCH OUT**] Ad-hoc or Playbook? Ad-hoc is for a task that's **quick and won't repeat**. The moment you feel the command will be reused or shared → move it to a Playbook immediately.

🔑 **EXAM** Memorize the structure: **Target** ← **Inventory (-i)** ← **Module (-m)** ← **Arguments (-a)** ← **Become**. Why no `-m` `command` in the uptime example? — it's the default Module.

▶ Full scenarios need an organized file — its language first: → [Chapter 10: YAML](#)

YAML Basics – The Language

YAML is a declarative language depending entirely on **Indentation** — not a programming language, just an orderly way to describe data.

[**WATCH OUT** • **Most important rule**] Never use **Tab** — always **two Spaces**. Mixing them is the #1 cause of failed YAML files, and the error appears far from its real line.

► **TASK [The first rule]** document start

Files always start with three dashes `---` — declaring the start of the YAML document:

```
---
```

► **TASK [The three data structures]** everything builds from these

① Key-Value

```
name: Ahmed
```

Key and value — the simplest structure.

② Dictionary

```
employee:
  name: Mohamed
  age: 30
  is_manager: true
```

Multiple keys under a parent key.

③ List

```
skills:
  - Linux
  - Ansible
  - Docker
```

A dash per item.

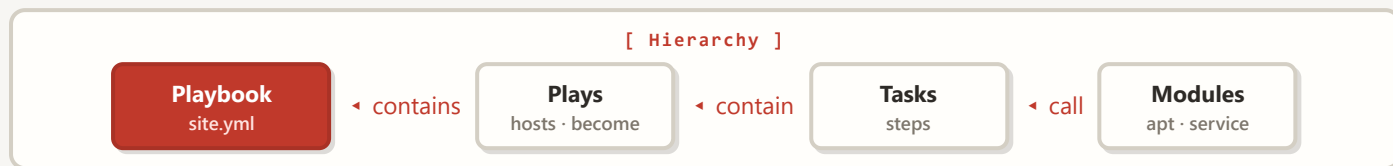
[**KEY IDEA**] The Playbook in the next chapter is nothing new — it's exactly a **List of Dictionaries**: each Play is an item in a List, each Task inside is a Dictionary. Understand these three structures and you understand all of Ansible.

🎓 **EXAM** The three golden rules: ① **Start with** `---` · ② **Spaces, never Tabs** · ③ **Three structures only: Key-Value, Dictionaries, Lists.**

🔗 Language in hand — your first real full Playbook: → [Chapter 11](#)

Playbooks – Write & Run

A **Playbook** is a YAML file containing a set of **Plays**, and each Play contains a set of **Tasks** — executing a complete automation scenario against a specific server group. This is Ansible's beating heart.



▶ TASK [Example: Playbook to install & run Apache] create site.yml

```

---
- name: Install and Start Apache Web Server
  hosts: web
  become: yes

  tasks:
    - name: Install Apache Package
      apt:
        name: apache2
        state: present

    - name: Ensure Apache Service is Running and Enabled
      service:
        name: apache2
        state: started
        enabled: yes
  
```

▶ TASK [Running the Playbook] execute it

```

$ ansible-playbook -i hosts.ini site.yml
PLAY [Install and Start Apache Web Server]
TASK [Gathering Facts] ok: [192.168.1.101]
TASK [Install Apache Package] changed: [192.168.1.101]
TASK [Ensure Apache Service is Running and Enabled] changed: [192.168.1.101]
PLAY RECAP
192.168.1.101 : ok=3 changed=2 failed=0
  
```

[**DEBUG · Line by line**] `hosts: web` → target the web group · `become: yes` → root privileges · every Task = descriptive **Name + Module + desired State**. We don't say "install" — we say "make it present."

[**PROOF**] **Run it a second time.** Result: `changed=0`, every Task `ok` — Idempotency alive before your eyes. ♻️

▶ Installed and running — clean removal and result reading: → [Chapter 11-b](#)

Playbooks – Removal & Results

The same power that installs also removes — by flipping the State. Then we master the three result indicators.

► TASK [Playbook to stop & remove Apache] reverse order

```
---
- name: Stop and Remove Apache Web Server
  hosts: web
  become: yes

  tasks:
    - name: Stop Apache Service
      service:
        name: apache2
        state: stopped

    - name: Remove Apache Package
      apt:
        name: apache2
        state: absent
```

[KEY IDEA] Reverse logical order: **stop the Service first**, then remove the Package — deleting a package while its service runs = unpredictable behavior. The whole secret is in `state`: `present / absent / started / stopped` — four values that rule the world.

► TASK [Result indicators] the three colors

Indicator	Color	Meaning
ok	Green	The server is already in the Desired State — no change required.
changed	Yellow	The Change was applied successfully.
failed	Red	An Error occurred — execution stopped on that server.

[RESCUED • If you see failed] Read the red Task name first — **which step** failed. Then the line under it — **why** (network? permissions? wrong package name?). 90% of failures are solved from these lines alone.

🎓 EXAM ok vs changed is **the core of Ansible's philosophy**: ok = state already satisfied (Idempotency working), changed = real work done now. Reading failed starts from the Task name + the Error Message below.

🔗 One file did a full scenario — but as projects grow it becomes a monster. Time for the deep dive: → [Chapter 12: Roles](#)

Roles – The Concept & Anatomy

As automation projects grow, keeping all tasks and instructions in one Playbook file becomes **hard to maintain**. **Ansible Roles** give you a standard folder structure to organize the project and split it into **Modular & Reusable** components.

[ASIDE • The analogy] Think of a Playbook as one long recipe scrawled on a single page — works for cooking once. A Role is a **recipe card box**: ingredients in one drawer, steps in another, serving notes in a third. Anyone can pick up the box and cook your dish, because everything lives where convention says it should. 📁

12.1 Why Roles exist — the problem they solve

Imagine a Playbook that installs a full web stack: packages, config files, templates, handlers to restart services, variables for different environments. At 300+ lines it becomes **unreadable, untestable, and unshareable**. Roles split this monolith by **concern**: "what runs" (tasks), "what gets copied" (files), "what gets rendered" (templates), "what varies" (vars). Each concern lives in its own folder — and Ansible knows where to find each one by convention, no wiring needed.

► TASK [The Role's folder structure] role anatomy

```
my_role/
├─ tasks/      # main.yml – the main tasks (the "what runs")
├─ handlers/   # conditional tasks – triggered by notify (12.b)
├─ templates/  # Jinja2 dynamic templates (12.b)
├─ files/      # static files – copied as-is
├─ vars/       # variables (high precedence – don't override easily)
├─ defaults/   # default variables (low precedence – made to be overridden)
└─ meta/       # role metadata & dependencies
```

Ansible finds each folder automatically when the Role runs — you never write paths like `my_role/tasks/main.yml` inside the Role itself.

12.2 vars/ vs defaults/ — the folder everyone mixes up

	<i>defaults/</i>	<i>vars/</i>
Purpose	Values meant to be overridden by whoever uses the Role	Values meant to stay fixed — internal to the Role
Precedence	Very low — easy to override	High — wins over most outside values
Example	<code>http_port: 80</code> (user may want 8080)	<code>_apache_service: apache2</code> (internal wiring)
Rule of thumb	"Settings the user may tune"	"Plumbing the Role needs to work"

[KEY IDEA] A Role is **Encapsulation**: instead of a 200-line Playbook, you call a Role by name — and everyone reading the project understands the structure instantly, because everyone knows this standard layout. That's the whole power of convention.

▶▶ The folders are just boxes until you fill them. Next: the two most important folders in practice — **templates/** and **handlers/** — with complete working examples: → [Chapter 12:b](#)

Templates & Handlers – Inside the Folders

Two Role folders confuse every beginner: **templates/** and **handlers/**. Here's both, made concrete with one realistic example — an Apache Role deploying a custom page.

12.3 Templates — files with holes for variables

A **template** is a text file with **Jinja2** placeholders — `{{ variable }}` — inside. At run time Ansible fills the holes with real values and writes the result to the server. Use it when a file should **differ per server** (hostname, port) instead of copying one identical static file everywhere.

templates/index.html.j2

```
<html><body>
<h1>Welcome to {{ inventory_hostname }}</h1>
<p>Port: {{ http_port }}</p>
<p>Maintained by {{ maintainer }}</p>
</body></html>
```

What lands on the server

```
<html><body>
<h1>Welcome to web-server-01</h1>
<p>Port: 8080</p>
<p>Maintained by Nabawy</p>
</body></html>
```

Same template → different output per server. `{{ inventory_hostname }}` is built-in (the server's name from the Inventory).

► TASK [tasks/main.yml — template + notify] render & trigger

```
---
- name: Install Apache
  apt: { name: apache2, state: present }
  notify: restart apache      # ← triggers the handler

- name: Deploy config from template
  template:
    src: ports.conf.j2
    dest: /etc/apache2/ports.conf
  notify: restart apache      # same handler — fires only ONCE
```

12.4 Handlers — tasks that run only when needed

A **handler** never runs on its own — it idles in `handlers/main.yml` until a task `notify` -s it. Even if **five tasks** notify the same handler in one run, it executes **exactly once** — **at the end**. Perfect for expensive operations like restarts.

handlers/main.yml

```
---
- name: restart apache
  service:
    name: apache2
    state: restarted
```

The mental model

A **doorman with a flag**: during the run, tasks only raise his flag (notify). He waits until every task finishes — then, if the flag is up, restarts Apache **once**. Flag never raised? He does nothing.

[**PROOF**] Config changed on 10 servers → Apache restarts **once per server**, not once per changed task. Config unchanged → **no restart at all**. Without handlers: restart blindly every run, or never restart. Handlers solve both.

[**WATCH OUT • Three traps**] **1.** `notify` matches the handler's **exact name** — a typo = silent no-op. **2.** Handlers only fire when a task reports `changed` — `ok` never triggers one. **3.** Quick rule: **identical** file everywhere → `files/` + `copy` · **varies per server** → `templates/` + `template` . (`.j2` is convention — keep it so readers see "has variables.")

🔥 **EXAM** Four facts: `template` → `template` module + Jinja2 · static file → `copy` · handlers run **once at the end** · they fire only on **changed**. These four lines answer most Role interview questions.

Ansible Galaxy – Use & Share Roles

Ansible Galaxy is the official open-source repository where the community publishes ready-made **Roles** and **Collections**. Instead of building an nginx role from scratch — search it, download it, adjust it.

[ASIDE] Galaxy is to Roles what an **app store is to apps** 📱: thousands of pre-built tools, rated and reviewed, installable in one command. And like an app store — check downloads and reviews before installing something into your infrastructure.

▶ TASK [Installing a Role from Galaxy] the commands

```
$ ansible-galaxy install geerlingguy.nginx
- geerlingguy.nginx (7.1.0) was installed successfully

$ ansible-galaxy install geerlingguy.nginx -p roles/ # custom path
$ ansible-galaxy install -r requirements.yml          # batch install
```

▶ TASK [requirements.yml — versioned, shareable] the professional way

```
# requirements.yml
- src: geerlingguy.nginx
  version: 7.1.0 # pin it — like package.json for infra
- src: geerlingguy.docker
  version: 7.0.2
```

[DEBUG • Why pin versions] Unpinned installs drift: the author updates and your automation silently changes behavior. A pinned `requirements.yml` in your repo = every teammate and every CI pipeline installs **exactly** the same roles. Reproducibility beats surprise.

12.7 Using an installed Role in your Playbook

```
---
- name: Set up web servers using a Galaxy role
  hosts: web
  become: yes
  roles:
    - geerlingguy.nginx # one line — brings tasks, templates,
                        # handlers, defaults... everything
```

[KEY IDEA] One line and Ansible loads the entire Role: tasks run, templates render, handlers arm, defaults fill unset values. To customize, override its **defaults** — e.g. `vars: nginx_default_port: 8080` in your Playbook. That's the defaults/-vs-vars/ split from Chapter 12 doing its job.

▶ TASK [Creating your own Role] scaffold in one command

```
$ ansible-galaxy init my_role
- my_role was created successfully
# ALL folders pre-made: tasks/ handlers/ templates/
# files/ vars/ defaults/ meta/ + README.md — fill & publish
```

1 Search galaxy.ansible.com

2 Install

3 Override defaults

4 Publish yours

[WATCH OUT] Not all Galaxy roles are equal — check **downloads**, **stars**, and maintenance. A 2019 role with 40 downloads isn't free automation. Trusted authors like `geerlingguy` have millions of downloads for a reason.

🔑 **EXAM** `install <role>` installs · `init <name>` scaffolds · `requirements.yml` pins versions · `roles:` in a Playbook consumes them. Four commands — the whole platform.



Test Yourself — 12 Questions

Cover the answers until you've solved all twelve. 10+ correct = practically ready.

1. What are the two primary domains Ansible is used for?

2. What's the only requirement on Linux Managed Nodes?

3. On which Node is Ansible installed?

4. Define Idempotency in one sentence with an example.

5. What's the difference between a template and a static file — and which modules deploy each?

6. Write the command to generate an SSH Key Pair of length 4096.

7. What's the difference between defaults/ and vars/ in a Role?

8. A Playbook run changes 3 config files, each notifying "restart apache." How many times does Apache restart — and when?

9. Write the command to install the geerlingguy.nginx role from Galaxy.

10. What does `ansible-galaxy init my_role` do?

11. What does `changed=0` in the PLAY RECAP mean after the second run?

 **Honor question:** The four features + three domains + the full "What is Ansible?" sentence — from memory.

[**DEBUG**] Questions cover chapters 1–12 in sequence — including the Roles deep dive (12, 12-b, 12-c). Any struggle → return to that chapter from the **TOC** before opening the answers.



Answer Key + Scoring

#	Answer	Why
1	Configuration Management + Infrastructure Provisioning	The two primary domains — the third: Network Automation
2	Python only	Pre-installed on most distros — no Agents
3	The Control Node only	Managed Nodes receive over SSH without installation
4	Running the same script repeatedly changes nothing unless a change is actually needed — example: Apache already installed → no change	Desired State satisfied → ok
5	Template = Jinja2 file with <code>{{ variables }}</code> , deployed with the <code>template</code> module — content varies per server · Static file = identical everywhere, deployed with <code>copy</code>	files/ for static · templates/ for dynamic (12-b)
6	<code>ssh-keygen -t rsa -b 4096</code>	-t the type (rsa) · -b the length (4096 bit)
7	defaults/ = values meant to be overridden by the role's user (low precedence) · vars/ = internal values meant to stay fixed (high precedence)	defaults = "user-tunable settings"; vars = "internal plumbing" (Ch 12)
8	Exactly once — at the end of the play. Even with 3 notify calls, handlers batch and fire a single restart per server	Handlers batch: notify raises a flag, the restart runs once (12-b)
9	<code>ansible-galaxy install geerlingguy.nginx</code>	Optionally add -p roles/ and a pinned version (12-c)
10	Scaffolds a complete Role skeleton — all folders pre-created (tasks/, handlers/, templates/, files/, vars/, defaults/, meta/) + README	init = instant standard structure (12-c)
11	The state was already satisfied — Idempotency worked 🔄	Same Playbook twice = zero Changes

[**PROOF** · 🔥 **Honor question**] The four features: **Agentless** · **Simple YAML Syntax** · **Declarative Approach** · **Idempotency** — the three domains: **Configuration Management** · **Infrastructure Provisioning** · **Network Automation**. Full sentence: an Agentless Automation tool over SSH, with YAML Declarative syntax for the Desired State, Idempotent, organized with Inventory and Playbooks, scaling with Roles, Templates, Handlers, and Galaxy. Answered from memory? You understand Ansible — you don't memorize it.

► TASK [Score yourself] scoring

10–12 · 🏆 **Practically ready**

Move on to building real Roles and a full automation project.

7–9 · 💪 **Solid**

Re-read the chapters you missed, then re-test.

<7 · 📺 **Replay**

Run the book's examples by hand on a real lab — doing sticks better than reading.

The Cheat Sheet — print & pin it

- ▶ **FULL FLOW:** keygen → ssh-copy-id → inventory → ad-hoc/playbook → recap
- ▶ **ARCHITECTURE:** Control Node —SSH→ Managed Nodes (Agentless · Python only)
- ▶ **STRUCTURE:** Playbook → Plays → Tasks → Modules · scale: Roles → Galaxy

[⚙️ Setup & Connection]

```
$ brew install ansible
$ sudo apt install ansible -y
$ pip3 install ansible
$ ssh-keygen -t rsa -b 4096
$ ssh-copy-id -i ~/.ssh/id_rsa.pub user@ip
$ ssh user@ip
```

[⚡ Ad-hoc Syntax]

```
ansible <group> -i hosts.ini -m <module> -a "args"
ansible all -i hosts.ini -m ping
ansible all -m apt -a "name=nginx state=present" --become
```

[🌌 Galaxy Commands]

```
$ ansible-galaxy install geerlingguy.nginx -p roles/
$ ansible-galaxy install -r requirements.yml
$ ansible-galaxy init my_role
```

[📄 Playbook Structure]

```
---
- name: scenario name
  hosts: web
  become: yes
  roles:
    - geerlingguy.nginx
  tasks:
    - name: task name
      apt:
        name: apache2
        state: present
        notify: restart apache

# run:
ansible-playbook -i hosts.ini site.yml
```

[🚨 Quick Facts]

template → {{ vars }}	copy → static files
handlers run once, at end	only on changed tasks
defaults/ = overridable	vars/ = internal
ok satisfied · changed applied · failed errored	

```
my_role/ tasks/ · handlers/ · templates/ · files/ · vars/ · defaults/ · meta/ # scaffold: ansible-galaxy init
```

Agentless

Declarative

Idempotent

Templates

Handlers

Roles + Galaxy

[**KEY IDEA · Closing line**] You write the Desired State once — Ansible enforces it across all your Managed Nodes, and touches nothing if no change is needed. When it grows: Roles organize it, Templates flex it, Handlers react to it, Galaxy shares it. ▶